

OPUS TECHNICAL MANUAL

Modular Cryptanalysis & Forensic Toolkit — Internal Architecture Guide

Version 1.0

Author: Greg

TITLE PAGE

OPUS TECHNICAL MANUAL

Internal Architecture & Developer Guide

Version 1.0

Author: Greg

Location: Wallkill, NY

TABLE OF CONTENTS

1. Introduction
 2. System Architecture
 3. Build & Installation
 4. Core Components
 5. Subsystems
 6. Thread Pool
 7. Logging & Progress Callbacks
 8. Data Flow & Pipelines
 9. File Outputs
 10. Extending Opus
 11. Directory Layout
 12. Developer Notes
 13. Glossary
 14. License
 15. Author
-

1. Introduction

Opus is a modular cryptanalysis and forensic extraction suite designed for extensibility, safety, and high-performance analysis.

This Technical Manual documents the internal architecture, subsystems, pipelines, and development patterns used throughout Opus.

This document is intended for:

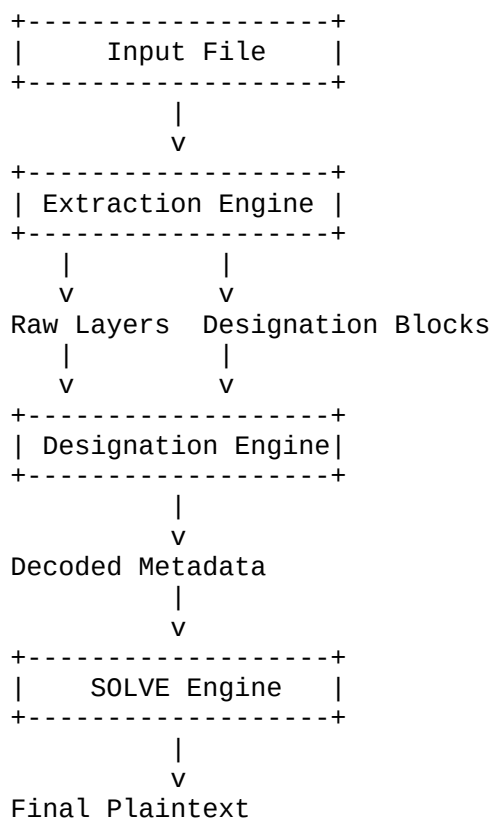
- Developers extending Opus
- Contributors adding new modules
- Researchers studying the architecture
- Anyone modifying internal components

The User Manual covers usage.

This manual covers *how Opus works internally*.

2. System Architecture

Opus is structured as a collection of modular engines connected through a unified command interface.



Key architectural principles:

- **Modularity** — each subsystem is isolated
- **Safety** — explicit destructive operations
- **Extensibility** — new engines can be added easily
- **Determinism** — reproducible solver behavior
- **Parallelism** — thread pool for long-running tasks

3. Build & Installation

3.1 Requirements

- GCC or Clang
- POSIX environment (Linux, macOS, WSL)
- pthreads
- math library (-lm)

3.2 Build Command

```
gcc -std=c11 -O3 -Wall -Wextra -lpthread -lm -o Opus *.c
```

3.3 Optional Components

- ncurses (for TUI mode)
- GMP (for RSA modules, if enabled)

4. Core Components

Opus is composed of several major modules:

- `Opus.c` — entry point, REPL, command dispatch
- `task_pool.c` — thread pool
- `solve.c` — modern substitution solver
- `solver.c` — legacy hillclimber
- `opus_extract.c` — layer extraction
- `opus_extract_designations.c` — designation extraction
- `opus_designation_*` — designation decoding and analysis
- `mutation.c` — key mutation logic
- `score.c` — quadgram scoring
- `state.c` — solver state management
- `logging.c` — progress callbacks
- `rng.c` — deterministic random number generator

5. Subsystems

5.1 Extraction Engine

Files:

- `opus_extract.c`
- `opus_extract_designations.c`

Responsibilities:

- Scan input files for embedded layers
- Extract raw binary blocks
- Identify designation structures
- Produce intermediate artifacts

Pipeline:

Input → Layer Scanner → Layer Extractor → Output Layers

5.2 Designation Engine

Files:

- `opus_designation_solve.c`
- `opus_designation_decoder.c`
- `opus_designation_analyze.c`

Responsibilities:

- Decode structured metadata
- Solve designation puzzles
- Produce human-readable summaries

Pipeline:

Designation Block → Decoder → Analyzer → Output

5.3 SOLVE Engine (Modern Solver)

Files:

- `solve.c`
- `mutation.c`
- `score.c`
- `logging.c`
- `state.c`
- `rng.c`

Responsibilities:

- Mutation-based key search
- Quadgram scoring
- Deterministic RNG

- Logging and progress callbacks
- Multi-threaded execution

Pipeline:

Ciphertext → Key Generator → Mutation Loop → Scoring → Best Key → Plaintext

5.4 Legacy Hillclimber

File:

- `solver.c`

A standalone quadgram hillclimber with simulated annealing.

Used for:

- Quick testing
- Comparison against the modern solver
- Simple monoalphabetic ciphers

6. Thread Pool

File:

- `task_pool.c`

Responsibilities:

- Parallel task execution
- Graceful cancellation
- Shutdown coordination
- Non-blocking solver operations

Pipeline:

Tasks → Queue → Worker Threads → Results

7. Logging & Progress Callbacks

Opus supports UI callbacks for real-time solver updates.

Callback registration:

```
solver_set_progress_cb(cb, user);
```

Progress messages include:

- iteration
- score

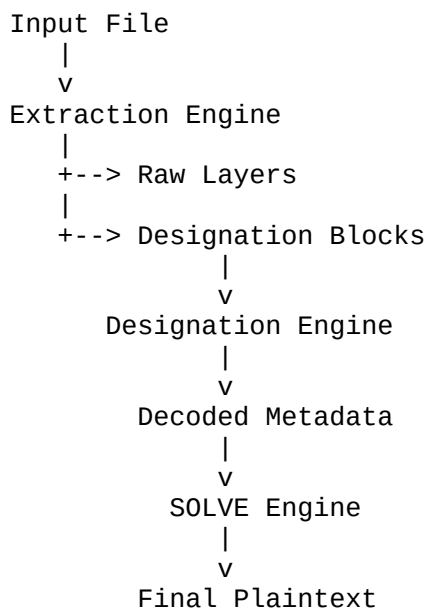
- best score
- key preview
- plaintext preview

These messages are consumed by:

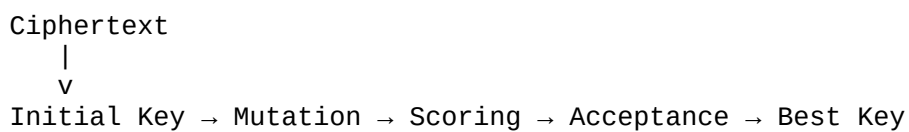
- CLI REPL
- TUI (optional)
- External integrations

8. Data Flow & Pipelines

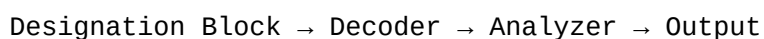
8.1 Full Pipeline



8.2 Solver Pipeline



8.3 Designation Pipeline



9. File Outputs

File	Description
solved.txt	Final plaintext output
solved_preview.txt	Preview after each restart
cipher.txt	Ciphertext input
english_quadgrams.txt	Quadgram scoring model

10. Extending Opus

10.1 Adding New Solvers

Extend:

- `solve.c`
- `mutation.c`
- `score.c`

Steps:

1. Implement new mutation or scoring logic
2. Register solver in command dispatch
3. Add progress callbacks
4. Document in the manual

10.2 Adding New Extraction Formats

Extend:

- `opus_extract.c`
- `opus_extract_designations.c`

10.3 Adding New Designation Types

Extend:

- `opus_designation_decoder.c`
- `opus_designation_analyze.c`

11. Directory Layout

Recommended structure:

```
/src
  Opus.c
  solve.c
  solver.c
  mutation.c
  score.c
  logging.c
  state.c
  rng.c
  task_pool.c
  opus_extract.c
  opus_extract_designations.c
  opus_designation_solve.c
  opus_designation_decoder.c
  opus_designation_analyze.c
```

Build script example:

```
#!/bin/sh
gcc -std=c11 -O3 -Wall -Wextra -lpthread -lm -o Opus *.c
```

12. Developer Notes

- Keep modules isolated
- Avoid global state where possible
- Maintain deterministic solver behavior
- Use explicit error handling
- Document new modules in this manual
- Follow existing naming conventions

13. Glossary

+-----+-----+	
Term	Definition
+-----+-----+	
Layer	Extracted data block
Designation	Structured metadata block
Quadgram	Four-letter scoring unit
Mutation	Key modification step
Annealing	Acceptance of worse solutions
Plaintext	Decoded output
+-----+-----+	

14. License

MIT License (or your preferred license).

15. Author

Greg — inventive cryptanalyst, systems builder, and creator of Opus.

=====
END OF DOCUMENT
=====